# Storage format of converted binary data used by ToBinary and FromBinary

Abraham Sharnoff
Document last updated on 2026-09-25

## Contents

Prelude ..... 3
1: Number Primitives ..... 3
1.a: Important distinction about usize and isize ..... 4
2: char ..... 4
3: bool ..... 4
4: String ..... 4
5: &str ..... 4
6: Vec<T> ..... 4
7: VecDeque<T> ..... 4
8: LinkedList<T> ..... 4
9: HashSet<T> ..... 4
9.a: With base hasher ..... 4
9.b: With non base hasher S ..... 4
10: HashMap<K, V> ..... 5
10.a: With base hasher ..... 5
10.b: With non base hasher S ..... 5
11: BinaryHeap<T> ..... 5
12: BTreeSet<T> ..... 5
13: BTreeMap<K, V> ..... 5
14: std::alloc::Layout ..... 5
15: Option<T> ..... 5
16: Result<T, E> ..... 5
17: () ..... 5
18: (T, U) ..... 5
19: (T, U, I) ..... 5
20: (T, U, I, O) ..... 6
21: std::cmp::Ordering ..... 6
22: Box<T> ..... 6
23: Ipv4Addr ..... 6
24: Ipv6Addr ..... 6
25: IpAddr ..... 6
26: SocketAddrV4 ..... 6
27: SocketAddrV6 ..... 6
28: SocketAddr ..... 6
29: std::io::ErrorKind ..... 7
30: std::ops::Bound<T> ..... 7

31: std::ops::Range<T> ..... 7
32: std::ops::RangeFrom<T> ..... 7
33: std::ops::RangeInclusive<T> ..... 7
34: std::ops::RangeTo<T> ..... 7
35: std::ops::RangeToInclusive<T> ..... 7
36: Duration ..... 7
37: SystemTime ..... 7
38: BacktraceStatus ..... 7
39: Cell<T> ..... 8
40: OnceCell<T> ..... 8
41: UnsafeCell<T> ..... 8
42: CString ..... 8
43: PhantomData<T> ..... 8
44: PhantomPinned<T> ..... 8
45: ManuallyDrop<T> ..... 8
46: std::ops::ControlFlow<B, C> ..... 8
47: std::process::ExitCode ..... 8
48: Mutex<T> ..... 8
49: std::sync::atomic::Ordering ..... 8

**Prelude**

If I say that something is stored as a type T, then look at that type's section to figure out how it is represented in binary.

If I say that something is stored as [T] then it is being stored as a slice of T, meaning that it is consecutive T bytes without any gaps and follows the same definition rules as for storing T.

If I say that something is stored as [T; N] then it is being stored as an array of T with length N meaning that it is equivalent to a slice but has a consistent and known length.

If I am going to describe how to store an enum then it will be using the following syntax:

- [variant]: [format]
- [variant]: [format]
- [variant]: [format]

Know that only one of those paths will be what gets saved.

If I am feeling particularly lazy and need to represent how a struct gets saved then I may use a table as such:

| [field]: [format] |
| --- |
| [field]: [format] |
| [field]: [format] |

Know that ALL of the fields will be saved. They will be saved in that specific order.

**1: Number Primitives**

All numbers are stored as [u8; N] where N is the number of bytes in that number primitive. So a u8 will have an N of 1 and a u16 will have an N of 2.

The first weird part is with floats because they are technically converted to the unsigned integer of the same before writing, so an f32 gets turned into a u32 first. But this doesn't actually matter because it leaves the bytes untouched in little endian order.

Which brings be to weird part number 2, all integers are stored in little endian byte order, which usually means no operations are done because computers generally store them in little endian byte order but it is good to enforce it just in case.

All of the NonZero number variants are stored as their zeroable counterparts.

All of the Atomic number variants are stored as there normal counterparts and when writing are read with Acquire ordering.

### 1.a: Important distinction about usize and isize

usize and isize are stored as though they were their 64 bit counterparts by default but if the "dyn_binary" feature flag is set then they are instead stored in their native size.

### 2: char

chars are stored as u32, don't ask me why. That's just what I can convert them to and from.

### 3: bool

- false: 0_u8
- true: 1_u8

### 4: String

| len: usize |
| --- |
| bytes of data: [u8] |

### 5: &str

It only implements ToBinary and does so in the same way as String.

### 6: Vec<T>

| len: usize |
| --- |
| contents: [T] |

Any time something says that it is stored the same as a Vec, gets converted to and from a Vec before and after being in binary, usually through a to_vec method.

### 7: VecDeque<T>

Same as Vec

### 8: LinkedList<T>

Same as Vec

### 9: HashSet<T>

### 9.a: With base hasher

Same as Vec

### 9.b: With non base hasher S

| len: usize |
| --- |
| hasher: S |
| contents: [T] |

The way we are getting the contents is iterating, not getting the raw bytes.

**10: HashMap<K, V>**

**10.a: With base hasher**
Same as a Vec of (K, V)

**10.b: With non base hasher S**

| len; usize |
|---|
| hasher: S |
| contents: [(K, V)] |

**11: BinaryHeap<T>**
Same as Vec

**12: BTreeSet<T>**
Same as Vec

**13: BTreeMap<K, V>**
Same as Vec of (K, V)

**14: std::alloc::Layout**

| size: usize |
|---|
| align: usize |

**15: Option<T>**
- Some(T): (true, T)
- None: false

**16: Result<T, E>**
- Ok(T): (false, T)
- Err(E): (true, E)

**17: ()**
No op

**18: (T, U)**

| T |
|---|
| U |

**19: (T, U, I)**

| T |
|---|

| U |
| --- |
| I |

**20: (T, U, I, O)**

| T |
| --- |
| U |
| I |
| O |

**21: std::cmp::Ordering**

- Equal: 0
- Less: 1
- Greater: 2

All as u8

**22: Box<T>**

Same as T

**23: Ipv4Addr**

Stored as u32

**24: Ipv6Addr**

Stored as u128

**25: IpAddr**

- V4: (false, Ipv4Addr)
- V6: (true, Ipv6Addr)

**26: SocketAddrV4**

| ip: Ipv4Addr |
| --- |
| port: u16 |

**27: SocketAddrV6**

| ip: Ipv6Addr |
| --- |
| port: u16 |
| flowinfo: u32 |
| scope_id: u32 |

**28: SocketAddr**

- V4: (false, SocketAddrV4)

- V6: (true, SocketAddrV6)

**29: std::io::ErrorKind**
I'm not typing all of that again, go to from_binary.rs and search for "ErrorKind" if you want to know how it works.

**30: std::ops::Bound<T>**
- Excluded(point: T): (0, T),
- Included(point: T): (1, T),
- Unbounded: 2

Discriminant is u8

**31: std::ops::Range<T>**

| start: T |
| --- |
| end: T |

**32: std::ops::RangeFrom<T>**
Stored as T

**33: std::ops::RangeInclusive<T>**

| start: T |
| --- |
| end: T |

**34: std::ops::RangeTo<T>**
Stored as T

**35: std::ops::RangeToInclusive<T>**
Stored as T

**36: Duration**

| secs: u64 |
| --- |
| subsec_nanos: u32 |

**37: SystemTime**
Stored as Duration since SystemTime::UNIX_EPOCH

**38: BacktraceStatus**
- Captured: 0
- Disabled: 1
- Unsupported: 2

Discriminant is u8.

If your BacktraceStatus has more variants then this code is no longer fully valid.

**39: Cell<T>**
Stored as T

**40: OnceCell<T>**
Stored as Option<T>

**41: UnsafeCell<T>**
Stored as T

**42: CString**
Stored as Vec<T>

**43: PhantomData<T>**
No op

**44: PhantomPinned<T>**
No op

**45: ManuallyDrop<T>**
Same as T

**46: std::ops::ControlFlow<B, C>**
- Break(value: B): (false, value),
- Continue(value: C): (true, value)

**47: std::process::ExitCode**
Stored as u8

**48: Mutex<T>**
Stored as <T>. Does acquire lock in order to write and will return Err if poisoned.

**49: std::sync::atomic::Ordering**
- AcqRel: 0
- Acquire: 1
- Relaxed: 2
- Release: 3
- SeqCst: 4

Discriminant is u8